Classification Algorithm:

SVM Grid:

```
The f1_macro value for best parameter {'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}: 0.9924946382
275899
```

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 1 81]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```
from sklearn.metrics import roc_auc_score


roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
np.float64(1.0)
```

In SVM got 0.99 accuracy

Decision Tree Grid:

```
The f1_macro value for best parameter {'criterion': 'gini', 'max_features': 'log2', 'splitter': 'ra
ndom'}: 0.97748391468277
```

```
]: print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[50  1]
 [ 2 80]]
```

```
]: print("The report:\n",clf_report)
```

```
The report:
              precision    recall  f1-score   support

           0       0.96      0.98      0.97        51
           1       0.99      0.98      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133
```

```
]: from sklearn.metrics import roc_auc_score


roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
]: np.float64(0.978000956480153)
```

In Decision Tree Grid:

Got 0.98 accuracy

Random Forest Grid

```
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)
```

```
The f1_macro value for best parameter {'criterion': 'entropy', 'max_features': 'log2', 'n_estimator
s': 10}: 0.97748391468277
```

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[50  1]
 [ 2 80]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.96      0.98      0.97        51
           1       0.99      0.98      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133
```

```
from sklearn.metrics import roc_auc_score


roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
np.float64(0.999402199904352)
```

In RandomForest Grid:

Got 0.98 accuracy

LogisticRegression Grid:

```
The f1_macro value for best parameter {'penalty': 'l2', 'solver': 'sag'}: 0.9924946382275899
```

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 1 81]]
```

```
print("The report:\n",clf_report)
```

```
The report:
               precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```
from sklearn.metrics import roc_auc_score


roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
np.float64(1.0)
```

In Logistic Regression Grid:

Got 0.99 Accuracy

KNN Grid:

```
The f1_macro value for best parameter {'algorithm': 'auto', 'metric': 'minkowski', 'n_neighbors':
5, 'weights': 'distance'}: 0.940494593126172
```

```python
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 8 74]]
```

```python
print("The report:\n",clf_report)
```

```
The report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```python
from sklearn.metrics import roc_auc_score


roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])
```

```
np.float64(1.0)
```

In KNN Grid:

Got 0.99 accuracy